

Collections Framework

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor (CSE), PIT
Parul University

1.	Introduction to Functional Programming.....	1
2.	Summary of Key Concepts.....	3
3.	References.....	3

10.1.1 Introduction to Functional Programming

◆ 1. What is Functional Programming?

- **Functional programming (FP)** is a programming paradigm where computation is treated as the evaluation of mathematical functions.
- In Java, FP became mainstream with **Java 8**, through features like:
 - **Lambda expressions**
 - **Method references**
 - **Streams API**
 - **Functional interfaces**

🔍 Key Characteristics:

- **Immutability:** Avoids changing state and mutable data.
- **First-class functions:** Functions can be passed as parameters.
- **Pure functions:** Output depends only on input, no side effects.

◆ Functional Interfaces

A **functional interface** is an interface that has **only one abstract method**.

📄 Common Built-in Functional Interfaces (from java.util.function):

Interface	Description	Method Signature
Predicate<T>	Takes input and returns boolean	boolean test(T t)
Function<T, R>	Takes input and returns result	R apply(T t)
Consumer<T>	Takes input, returns nothing	void accept(T t)
Supplier<T>	Takes no input, returns result	T get()
BiFunction<T,U,R>	Takes 2 inputs, returns result	R apply(T t, U u)

Example:

```
Predicate<Integer> isEven = n -> n % 2 == 0;
System.out.println(isEven.test(4)); // Output: true
```

◆ Lambda Expressions

A **lambda expression** is a concise way to represent an anonymous function.

🔗 Syntax:

`(parameter) -> expression`

Examples:

```
Consumer<String> greet = name -> System.out.println("Hello " + name);  
greet.accept("Alice"); // Output: Hello Alice
```

```
Function<Integer, Integer> square = x -> x * x;  
System.out.println(square.apply(5)); // Output: 25
```

◆ Method References

Method reference is a shorthand notation of a lambda expression to call a method.

Syntax:

`ClassName::methodName`

Types:

- **Static method:** `Math::abs`
- **Instance method of an object:** `System.out::println`
- **Constructor reference:** `ArrayList::new`

```
List<String> names = Arrays.asList("John", "Jane", "Doe");  
names.forEach(System.out::println);
```

◆ Streams API

- Introduced in **Java 8**, the **Streams API** allows you to process data in a **declarative and functional style**.
- It enables **bulk operations** on collections (like filter, map, reduce) using pipelines.

"Stream represents a sequence of elements supporting sequential and parallel aggregate operations."

- **Key Features of Streams**
 - **Functional-style programming** with operations like map, filter, reduce.
 - **Laziness-seeking:** Operations are evaluated only when necessary.
 - **Pipelining:** Operations are chained to form a stream pipeline.
 - **Internal iteration:** Java handles iteration behind the scenes (vs. external loops).
- **Creating Streams**
 - **From Collections:**

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");  
Stream<String> stream = names.stream();
```

- **From Arrays:**

```
int[] numbers = {1, 2, 3, 4};  
IntStream intStream = Arrays.stream(numbers);
```

Using Stream.of():

```
Stream<String> stream = Stream.of("a", "b", "c");
```

➤ Summary of Key Concepts:

- Functional programming in Java enhances readability, testability, and conciseness.
- Functional interfaces enable lambda expressions.
- Lambdas and method references simplify the use of one-method interfaces.
- Widely used in Collections (Streams), Event Handling, and APIs.
- Streams process data in a functional, declarative, and chainable way.
- Intermediate operations are lazy and terminal operations trigger execution.
- Stream pipelines = **source** → **intermediate ops** → **terminal op**.
- Improves readability, performance (with parallelism), and code conciseness.

➤ References:

- Oracle Java Documentation – Functional Interfaces
🔗 <https://docs.oracle.com/javase/8/docs/api/java/util/function/package-summary.html>
- GeeksforGeeks – Functional Programming in Java
🔗 <https://www.geeksforgeeks.org/functional-programming-in-java/>
- Baeldung – Guide to Functional Interfaces
🔗 <https://www.baeldung.com/java-functional-interfaces>
- Oracle Java Tutorial – Lambda Expressions
🔗 <https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html>
- Oracle Java Streams API Tutorial
🔗 <https://docs.oracle.com/javase/8/docs/api/java/util/stream/package-summary.html>
- GeeksforGeeks – Java Stream API
🔗 <https://www.geeksforgeeks.org/stream-in-java/>
- JavaPoint – Java 8 Stream Tutorial
🔗 <https://www.javatpoint.com/java-8-stream>

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | **GRADE**



    
<https://paruluniversity.ac.in/>